LLMs

Complete Developer Study Notes

Concepts • Practical Examples • Projects • Interview Preparation

Prepared by: Rahul Yadav

20 Pages

Rahul Yadav

1. What are LLMs ? (Overview)

1.1 | Definition

→ Large Language Models (LLMs) are AI models trained on massive amounts of text data to understand, generate and work with human language.

→ They learn patterns, context and relationships between words to perform a wide range of NLP tasks.

→ LLMs are based on the Transformer architecture and are trained using self-supervised learning.

1.2 | Key Characteristics

1. Trained on large-scale text data
2. Based on Transformer architecture
3. Understand context and generate human-like text
4. Can perform multiple language tasks
5. Show emergent capabilities with scale
6. Can follow instructions (instruction tuning)
7. Can be adapted for specific use cases (fine-tuning / RAG / agents)

1.3 | How LLMs Work ? (High Level)



1.4 | Example

Input Prompt:

Explain LLMs in simple words. → LLM (Trained Model) →

Generated Output:

"Large Language Models are AI models trained on huge amounts of text data to understand and generate human language. They can answer questions, summarize text, write code and perform many other tasks.

1.5 | Common Use Cases

1. Question Answering
2. Text Summarization
3. Text Generation (Blogs, Emails, Content)
4. Code Generation
5. Translation
6. Chatbots and Virtual Assistants
7. Document Analysis (PDF, DOC, etc.)
8. Data Extraction
9. RAG (Use your own data)
10. AI Agents and Automation

1.6 | Popular LLMs

1	OpenAI GPT-4 / GPT-4o	: OpenAI
2	Claude 3 / 3.5	: Anthropic
3	Gemini 1.5	: Google
4	Llama 3	: Meta
5	Mistral	: Mistral AI
6	Mixtral	: Mistral AI
7	Phi	: Microsoft
8	Qwen	: Alibaba
9	DeepSeek	: DeepSeek AI

1.7 | Key Takeaways

→ LLMs are powerful language models trained on large text data.

→ They use Transformer architecture and predict the next token.

→ They can perform many real-world tasks like QA, summarization, coding, etc.

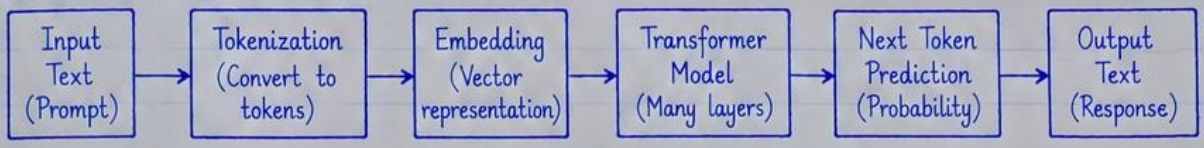
→ Capabilities improve with more data, better training and fine-tuning.

→ LLMs can be customized using fine-tuning, RAG or agents for specific needs.

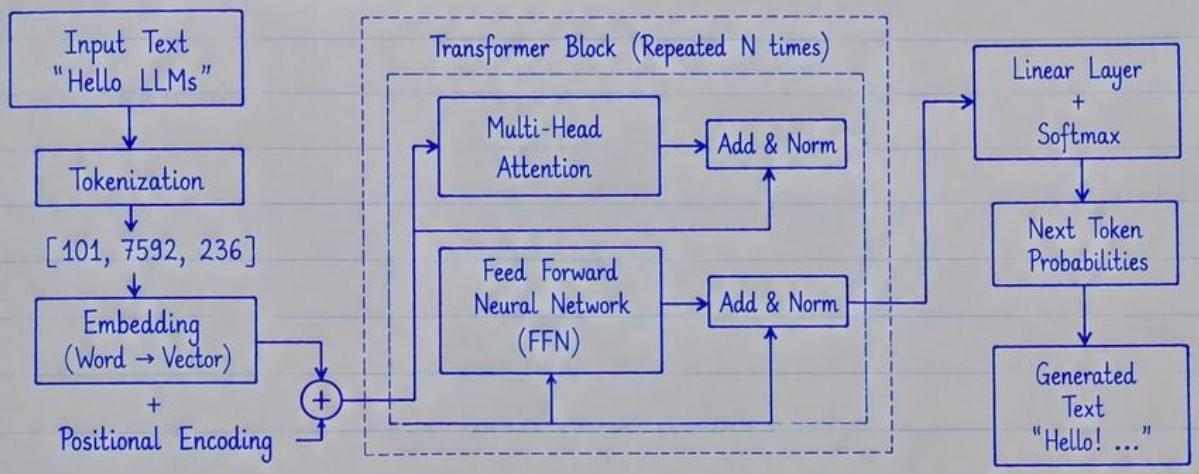
Rahul Yadav

2. How LLMs Work ? (Architecture)

2.1 High Level Flow



2.2 Detailed Architecture (Transformer)



2.3 Tokenization

- Converts input text into smaller units called tokens.
- Tokens can be words, subwords or characters.
- Common algorithms: BPE, WordPiece, SentencePiece.
- Example:
  Text: "I love LLMs"
  Tokens: ["I", "love", "LLM", "s"]

2.4 Embeddings & Positional Encoding

- Each token is converted into a dense vector (embedding).
- Embeddings capture the meaning of tokens.
- Positional Encoding adds information about the position of tokens in the sequence (since transformers don't have RNN).

Token Id	→	Embedding Vector (e.g. 768 dimensions)
101	→	[0.12, -0.03, 0.45, ...]
7592	→	[0.01, 0.67, -0.11, ...]
236	→	[0.28, -0.44, 0.09, ...]

2.5 Self-Attention (Key Idea)

- Allows the model to look at all tokens in the input and understand their relationships.
- Computes attention scores between tokens.
- Generates a weighted representation of tokens based on their relevance.
- Formula (simplified):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Query K = Key V = Value

d_k = dimension of keys/queries

2.6 Why Transformer Works Well ?

1. Handles long-range dependencies
2. Processes all tokens in parallel
3. Uses self-attention to capture context
4. Scales well with large data and model size
5. Can be pre-trained and fine-tuned easily

2.7 Summary

- Input text → tokens → embeddings
- Add positional encoding
- Pass through multiple transformer blocks (self-attention + feed forward)
- Predict next token using softmax
- Repeat to generate complete response

Rahul Yadav

3. Tokens, Tokenization and Vocabulary

3.1 What are Tokens ?

- Tokens are the basic units of text that LLMs understand.
- A token can be a word, subword, part of a word or even a single character.
- LLMs work with tokens instead of raw text.
- The model converts input text into a sequence of tokens (tokenization).

3.2 Examples of Tokens

Text	Possible Tokens (depends on tokenizer)
Hello	["Hello"] or ["Hel", "lo"]
I love AI	["I", " love", " AI"]
ChatGPT	["Chat", "GPT"] or ["ChatGPT"]
Unbelievable	["Un", "believ", "able"]
2024	["2024"] or ["20", "24"]
☺ (emoji)	["☺"] (often single token)
Code	["Code"]
Running	["Run", "ning"]

3.3 Tokenization Process

Input Text "I love LLMs" → Tokenizer (BPE / SentencePiece etc.) → Tokens ["I", " love", " LLM", "s"] → Token IDs [101, 428, 987, 32]

3.4 Common Tokenization Techniques

1. BPE (Byte Pair Encoding)
   - Most commonly used (GPT, Llama, etc.)
   - Merges frequent pairs of characters/bytes.
   - Handles new words effectively.
2. WordPiece (Used by BERT)
   - Similar to BPE.
   - Uses a fixed vocabulary and greedily matches the longest subword.
3. SentencePiece (Used by T5, Llama, etc.)
   - Works on raw text (no pre-tokenization).
   - Supports multiple languages well.
4. Unigram Language Model
   - Probabilistic approach.
   - Chooses the best tokenization based on likelihood.

3.5 Vocabulary

- Vocabulary is the set of all possible tokens that the model can understand.
- Each token has a unique ID.
- The vocabulary size can range from 20,000 to 200,000+ tokens.
- Larger vocabulary can handle more words but increases model size.
- Many tokenizers include special tokens like <s>, </s>, <pad>, <unk>, etc.

3.6 Special Tokens

Token	Purpose
<s>	Start of sequence (beginning)
</s>	End of sequence (end)
<pad>	Padding (make all inputs same length)
<unk>	Unknown token (not in vocabulary)
<mask>	Used in some models (e.g., BERT)
<bos>	Beginning of sentence
<eos>	End of sentence

3.7 Tokenization Example (Demo)

Text: "Generative AI is amazing!"

↓

Tokens: ["Generative", " AI", "is", " amazing", "!"]

↓

Token IDs (example): [3201, 428, 318, 987, 13]

3.8 Key Takeaways

1. LLMs process text as tokens, not characters or words.
2. Tokenization converts text into a sequence of tokens.
3. Different tokenization algorithms are used (BPE, WordPiece, SentencePiece, etc.).
4. Vocabulary defines all possible tokens.
5. Special tokens are used for model control.
6. Tokenization helps handle new words, multiple languages and reduces vocabulary size.
7. The same text can be tokenized differently using different tokenizers.

Rahul Yadav

4. Pre-training and Training Process

4.1 Overview

- Pre-training is the process of training LLMs on massive amounts of unlabeled text data using self-supervised learning.
- The model learns to predict the next token in a sequence.
- It learns general language understanding, world knowledge and reasoning abilities.
- After pre-training, the model can be further improved using fine-tuning, instruction tuning or alignment.

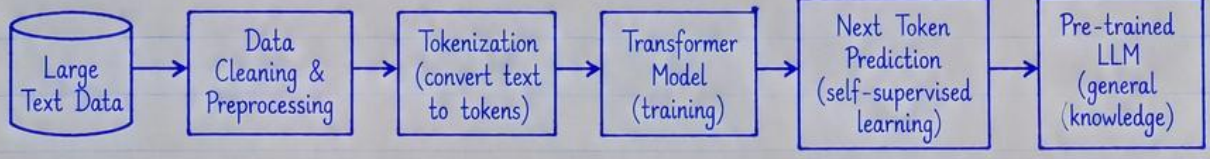
4.2 Pre-training Data

- Huge and diverse text data from multiple sources.
- Examples: books, articles, websites, code, forums, research papers, etc.
- Data is cleaned, filtered and deduplicated.
- High-quality and diverse data helps better performance.

Data Sources (Examples)



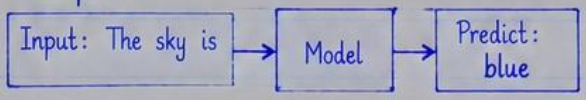
4.3 Pre-training Workflow



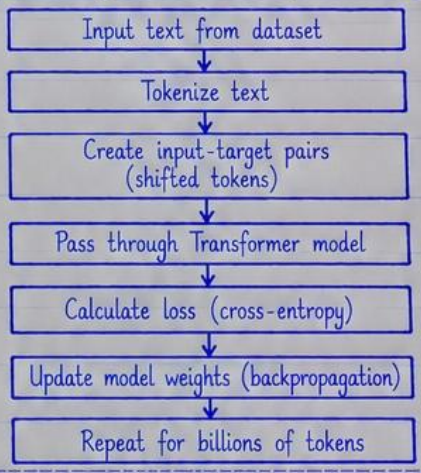
4.4 Objective - Next Token Prediction

- The model is trained to predict the next token given the previous tokens.
- This is called Causal Language Modeling.
- It uses a large corpus of text and learns statistical patterns in language.
- Training objective is to minimize the cross-entropy loss between predicted and actual next token.

Example:



4.5 Training Process (High Level)



4.6 Loss Function (Cross-Entropy Loss)

- Measures how different the predicted distribution is from the actual next token.
- The model predicts a probability distribution over the entire vocabulary.
- Cross-entropy loss is used to train the model.

Formula (Simplified):

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log P(y_i \mid x_1, \ldots, x_{i-1})$$

N = total number of tokens
y_i = actual next token
$P(y_i \mid x_1, \ldots, x_{i-1})$ = predicted probability

4.7 Important Concepts

1. **Self-Supervised Learning**
   No labeled data is required. Model creates its own labels (next token).
2. **Large Scale Training**
   Trained on billions of tokens using powerful GPUs.
3. **Long Context Windows**
   Modern LLMs can handle large context lengths (e.g., 8K, 32K, 128K tokens).
4. **Compute Intensive**
   Requires large computational resources and time.
5. **Emergent Abilities**
   With scale, models show reasoning, problem solving and other advanced capabilities.

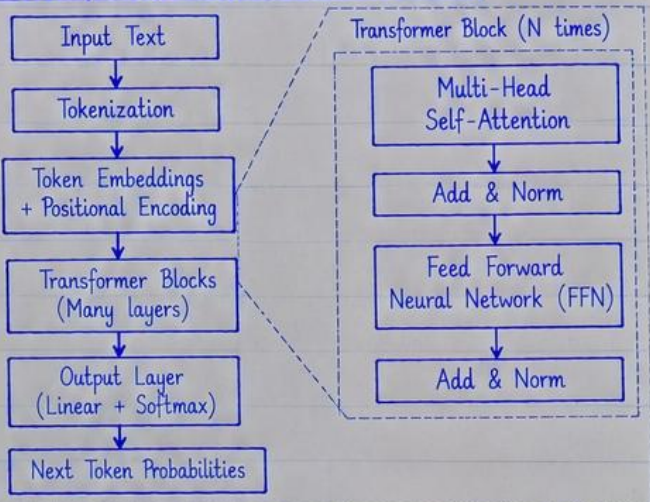
Rahul Yadav

5. Model Architectures (e.g., Transformer)

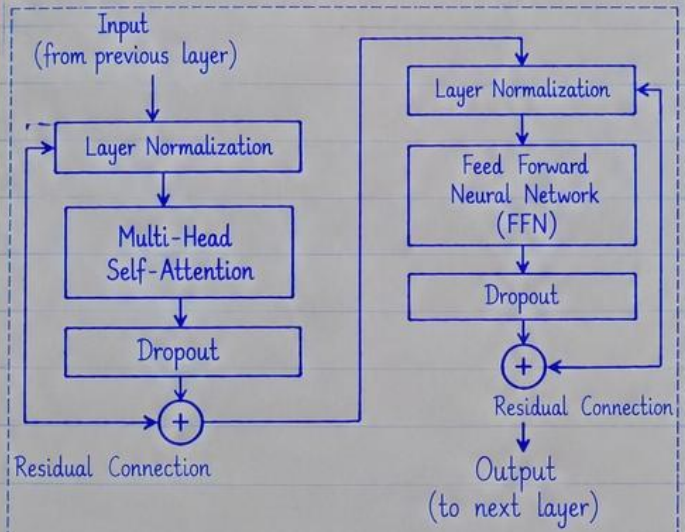
5.1 Overview

- Transformers are the core architecture behind most modern LLMs.
- They are designed to handle sequential data (text) using self-attention instead of recurrent networks (RNN / LSTM).
- Transformers can process all tokens in parallel, which makes them more efficient and effective for large-scale training.
- Popular LLMs like GPT, Llama, BERT etc. are based on Transformer architecture.

5.2 High Level Architecture



5.3 Transformer Block (Detailed)



5.4 Self-Attention (Key Idea)

- Self-attention allows the model to look at all tokens in the input and understand their relationships.
- Each token attends to every other token and computes a weighted representation.
- Helps in capturing long-range dependencies.
- Uses Query (Q), Key (K) and Value (V) vectors.

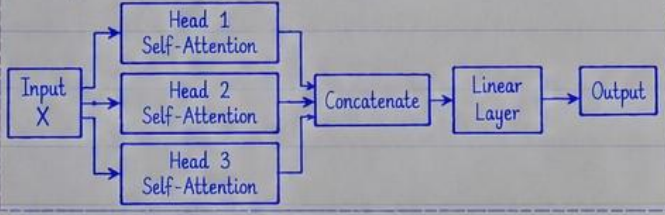
Self-Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Query matrix $(n \times d_k)$
K = Key matrix $(n \times d_k)$
V = Value matrix $(n \times d_v)$
d_k = dimension of keys / queries
n = number of tokens

5.5 Multi-Head Attention

- Uses multiple attention heads in parallel.
- Each head learns different patterns and relationships.
- Outputs from all heads are concatenated and passed through a linear layer.



5.6 Feed Forward Network (FFN)

- A simple 2-layer neural network with activation function (usually GELU).
- Applied to each token independently and in parallel.

FFN Formula

$$\text{FFN}(x) = \text{GELU}\left(xW_1 + b_1\right)W_2 + b_2$$

$W_1 : (d_{model} \times d_{ff})$
$W_2 : (d_{ff} \times d_{model})$
d_{ff} : usually $4 \times d_{model}$ (e.g. 4×4096)

5.7 Positional Encoding

- Transformers do not have a built-in sense of word order.
- Positional encoding adds information about the position of each token in the sequence.
- Can be learned or use fixed sinusoidal encoding.

5.8 Key Takeaways

1. Transformer is the most popular architecture for LLMs.
2. Uses self-attention and feed forward layers.
3. Can process tokens in parallel.
4. Handles long context and complex relationships.
5. Forms the base for models like GPT, Llama, BERT, etc.

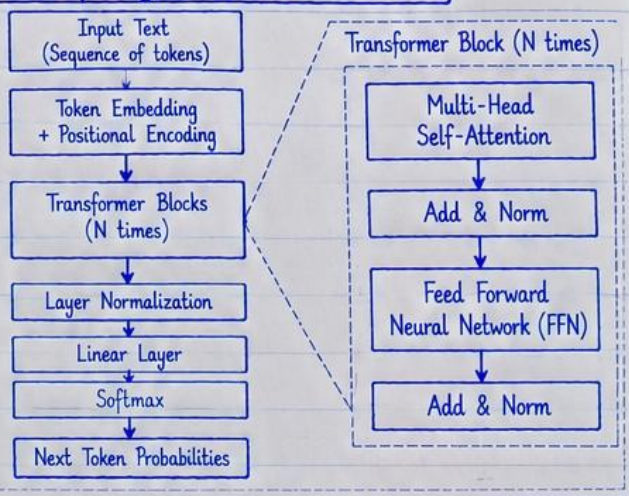
Rahul Yadav

5. Model Architectures (e.g., Transformer)

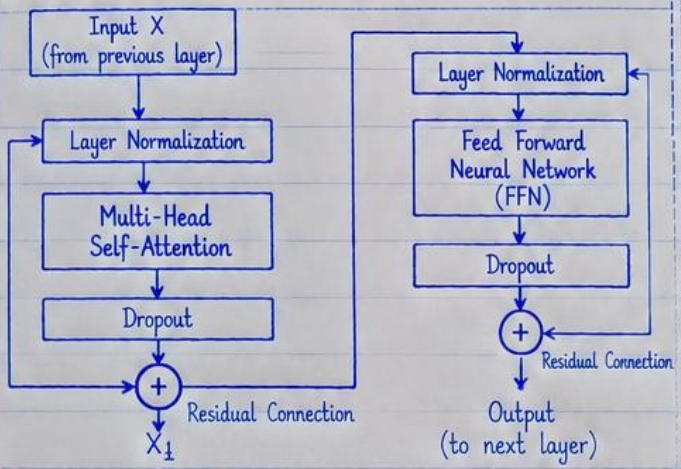
5.1 Overview

- Transformers are the core architecture behind most modern LLMs.
- Designed to handle sequential text data using self-attention instead of RNN/LSTM.
- Can process all tokens in parallel which makes training faster and more efficient.
- Scales well to large datasets and model sizes.
- Popular LLMs like GPT, Llama, BERT, etc. are based on Transformer architecture.

5.2 High Level Architecture

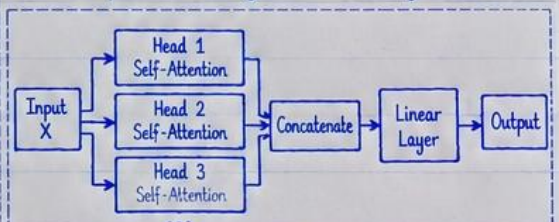


5.3 Transformer Block (Detailed)



5.4 Multi-Head Self-Attention

- Uses multiple attention heads in parallel.
- Each head learns different patterns and relationships.
- Helps the model to focus on different parts of the input sequence.
- Outputs from all heads are concatenated and passed through a linear layer.



5.5 Self-Attention (Scaled Dot-Product)

- Computes attention scores between all tokens.
- Measures how much each token should attend to every other token.
- Uses Query (Q), Key (K) and Value (V) vectors.
- Scaled by $\sqrt{d_k}$ to prevent large values.

Self-Attention Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q = Query matrix $(n \times d_k)$
K = Key matrix $(n \times d_k)$
V = Value matrix $(n \times d_v)$
d_k = dimension of keys/queries
n = number of tokens

5.6 Feed Forward Network (FFN)

- A simple 2-layer neural network applied independently to each token.
- Usually uses a hidden dimension larger than the model dimension.
- Activation function used is typically GELU.

FFN Formula:

$$\text{FFN}(x) = \text{GELU}\left(xW_1 + b_1\right)W_2 + b_2$$

W_1 : $(d_{model} \times d_{ff})$
W_2 : $(d_{ff} \times d_{model})$
d_{ff} : feed forward dimension (e.g. $4 \times d_{model}$)

5.7 Positional Encoding

- Transformers do not have a built-in sense of word order.
- Positional encoding adds information about the position of each token in the sequence.
- Can be learned or use fixed sinusoidal encoding.

5.8 Key Takeaways

1. Transformer is the most popular architecture for LLMs.
2. Uses self-attention and feed forward layers.
3. Can process tokens in parallel (faster training).
4. Handles long context and complex relationships.
5. Forms the base for models like GPT, Llama, BERT, etc.

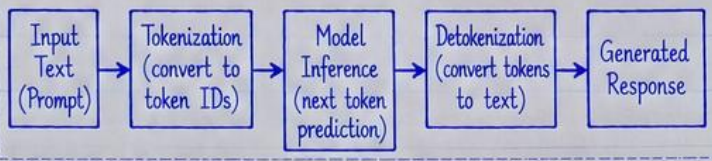
Rahul Yadav

6. Inference and Text Generation

6.1 What is Inference ?

- Inference is the process of using a trained LLM to generate a response for a given input (prompt).
- During inference, the model does not learn or update its weights.
- It uses the learned knowledge from training to predict the next token in a sequence.
- Inference can be done on a single GPU, multiple GPUs or with optimized engines for faster performance.

6.2 Inference Workflow



6.3 Next Token Prediction (Autoregressive)

- LLMs generate text one token at a time.
- Each next token is predicted based on previously generated tokens.
- The process continues until a stop condition is met (max tokens, end token, etc.).
- This is called autoregressive generation.

Example:

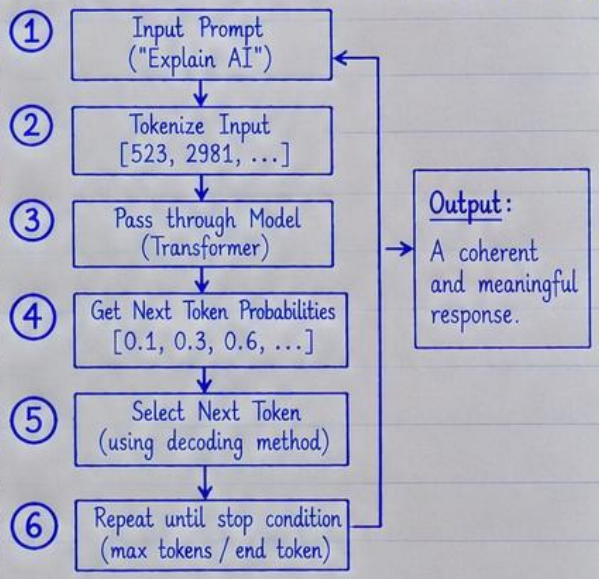
Prompt: "The sky is"

→ ? → sky → blue → is → blue

6.4 Inference vs Training

Feature	Training	Inference
Purpose	Learn model weights	Generate response using trained model
Data	Large labeled datasets	User input (prompt)
Weights Update	Yes	No
Compute Cost	Very high	Lower (but can be high for large models)
Time	Days/Weeks	Real-time (seconds)
Output	Trained model	Generated text

6.5 Text Generation Process (Step by Step)



6.6 Decoding Strategies

- Decoding strategy decides how the next token is selected from the probability distribution.

Strategy	How it works	Characteristics
Greedy Search	Picks the highest probability token	Fast, but may produce repetitive or less diverse text
Beam Search	Keeps top k sequences	Better quality, more compute
Top-k Sampling	Randomly selects from top k tokens	More diverse and creative
Top-p (Nucleus) Sampling	Selects from smallest set with cumulative probability p	More natural and coherent
Temperature Sampling	Controls randomness in selection	Low = deterministic High = more creative

6.7 Important Parameters

Parameter	Description	Example Value
max_tokens	Maximum number of tokens to generate	50, 100, 500
temperature	Controls randomness (0 = deterministic)	0.0 – 1.0 (e.g., 0.7)
top_k	Number of top tokens to consider	20, 50
top_p	Cumulative probability threshold	0.9, 0.95
repetition_penalty	Reduces repeated tokens	1.0 – 2.0

6.8 Example (Text Generation)

Prompt: "What is machine learning?"

Model Output:

Machine learning is a subset of artificial intelligence that enables computers to learn patterns from data and make predictions without being explicitly programmed.

6.9 Key Takeaways

1. Inference uses the trained model to generate text.
2. LLMs predict the next token in an autoregressive way.
3. Different decoding strategies control the output quality.
4. Parameters like temperature, top-k and top-p influence the response.
5. Inference can be optimized using techniques like quantization, caching and parallelization.

Rahul Yadav

7. Embeddings and Vector Representations

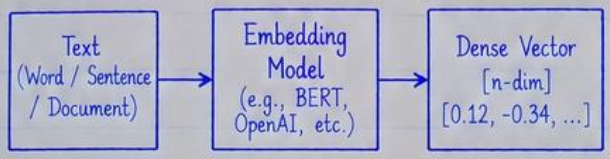
7.1 What are Embeddings ?

- Embeddings are numerical vector representations of text (words, sentences or documents).
- They capture the semantic meaning of text in a continuous vector space.
- Similar texts have similar vector representations (closer in the vector space).
- Embeddings help LLMs understand the context, similarity and relationships between words and sentences.

7.2 Why Embeddings are Important ?

1. Convert text into numerical vectors that machines can understand.
2. Capture semantic meaning (not just keywords).
3. Enable similarity search (e.g., in vector databases).
4. Used in retrieval augmented generation (RAG).
5. Help in clustering, classification, recommendation and search systems.

7.3 How Embeddings Work ?



7.4 Example: Embeddings for Similar Texts

Text 1: "I love machine learning"
Text 2: "I like machine learning"
Text 3: "The weather is nice today"

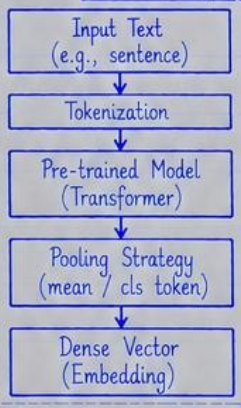
Embedding Vectors (3 dimensions for simplicity)	
Text 1:	[0.2, 0.7, -0.1]
Text 2:	[0.21, 0.69, -0.08]
Text 3:	[-0.4, 0.1, 0.9]

- Text 1 and Text 2 have similar vectors (close in vector space) because their meaning is similar.

7.5 Types of Embeddings

Type	Description	Example Models	Use Case
Word Embeddings	Vector for each word	Word2Vec, GloVe	Basic word representation
Contextual Embeddings	Vector depends on context	BERT, RoBERTa, DistilBERT	Better semantic understanding
Sentence Embeddings	Vector for entire sentence	Sentence-BERT, OpenAI Embeddings	Similarity search, clustering
Document Embeddings	Vector for entire document	BERT, Longformer	Search, RAG, classification
Multimodal Embeddings	Vector for images, audio, etc.	CLIP, OpenAI multi-modal	Image-text similarity

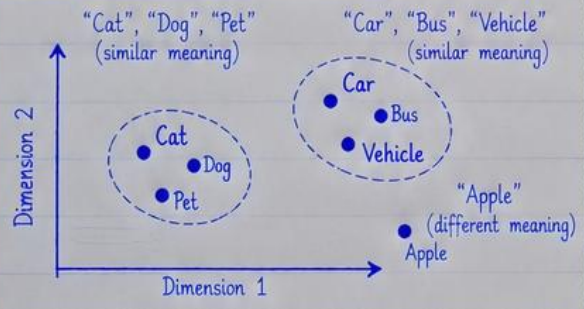
7.6 Embedding Model Architecture (High Level)



Pooling Strategies

- CLS Token
  Use the embedding of [CLS] token.
- Mean Pooling
  Average all token embeddings.
- Max Pooling
  Take max value across tokens.

7.7 Visual Representation (Vector Space)



7.8 Similarity Calculation

- To find how similar two texts are, we use cosine similarity between their vectors.

Cosine Similarity Formula:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

$A \cdot B$: dot product
$\|A\|$: magnitude of vector A
$\|B\|$: magnitude of vector B
θ : angle between vectors

- Value range: -1 to 1
- 1 → very similar
- 0 → no similarity
- -1 → completely opposite

7.9 Use Cases of Embeddings

1. Semantic search (find similar documents)
2. Retrieval Augmented Generation (RAG)
3. Clustering and topic modelling
4. Text classification (e.g., spam detection)
5. Recommendation systems
6. Question answering
7. Deduplication and similarity detection

7.10 Key Takeaways

1. Embeddings convert text into dense numerical vectors.
2. Similar texts have similar vectors in the vector space.
3. Different types: word, contextual, sentence and document embeddings.
4. Used in many real-world applications like search, RAG and classification.
5. Cosine similarity is commonly used to measure similarity.
6. Embedding models like BERT, Sentence-BERT and OpenAI provide high-quality representations.

Rahul Yadav

8. Fine-tuning LLMs

8.1 What is Fine-tuning ?

- Fine-tuning is the process of further training a pre-trained LLM on a smaller, domain-specific dataset.
- It helps the model adapt to specific tasks, formats, styles or domains.
- The base model already knows general knowledge, fine-tuning teaches new behaviors.
- It can be full fine-tuning or parameter-efficient fine-tuning (PEFT).

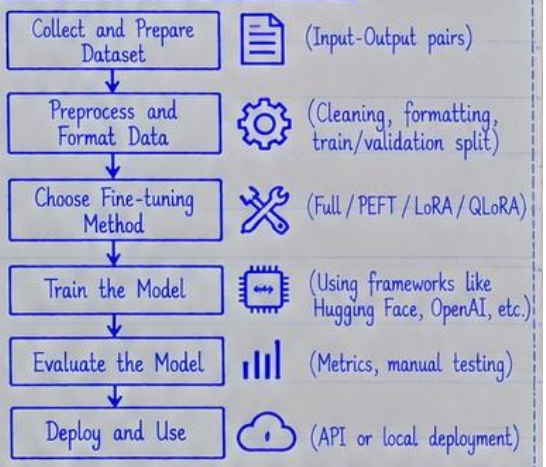
8.2 Why Fine-tuning ?

1. Improve performance on domain-specific tasks.
2. Teach custom behavior (e.g., JSON output, tone, format).
3. Reduce hallucinations in specific domain.
4. Requires fewer examples compared to prompt engineering for complex tasks.
5. More cost-effective than training from scratch.

8.3 Fine-tuning vs Prompting vs RAG

Feature	Prompting	RAG	Fine-tuning
Goal	Use model as-is	Provide external knowledge	Adapt model behavior
Data Needed	None	External documents	Labeled dataset
Training	No	No	Yes
Cost	Low	Medium	Medium to High
Best For	General tasks	Up-to-date information	Specific domain tasks and style
Model Update	Not required	Not required	Model weights updated
Example	"Explain AI"	"Use company docs to answer"	"Always respond in JSON format"

8.4 Fine-tuning Workflow



8.5 Types of Fine-tuning

Type	Description	Pros	Cons	Use Case
Full Fine-tuning	Update all model parameters	Best performance	High compute and cost	Large domain-specific tasks
PEFT (Parameter Efficient)	Update only small subset of parameters	Lower cost, faster training	Slightly lower performance	Most real-world tasks
LoRA (Low Rank Adaptation)	Add small trainable matrices to model layers	Very efficient, low memory	May need tuning	Popular and effective
QLoRA	Quantized LoRA (4-bit training)	Very low memory, run on single GPU	Slightly lower accuracy	Best for large models (e.g., Llama 2, 3)

8.6 Example: Fine-tuning Data Format

Input (Prompt)

```
{
    "instruction": "Extract customer name
      and account number from the text.",
    "input": "My name is Rahul Yadav
      and my account number is 123456789."
}
```

↓

Output (Expected Response)

```
{
    "customer_name": "Rahul Yadav",
    "account_number": "123456789"
}
```

8.7 Evaluation Metrics

Metric	Description	Use Case
Accuracy	Correctness of output	Classification, extraction
BLEU / ROUGE	Text similarity with reference	Text generation
Perplexity	How well the model predicts text	Language modeling
Human Evaluation	Manual quality check	Complex tasks (chat, style)
Task-specific Metrics	Custom metrics (e.g., F1, Exact Match)	Domain-specific tasks

8.8 Tools and Frameworks

- Hugging Face Transformers (Trainer API)
- PEFT library (LoRA, QLoRA)
- Datasets library (for data loading)
- BitsAndBytes (for 4-bit training)
- OpenAI Fine-tuning API
- DeepSpeed / Accelerate (for large models)

8.9 Key Takeaways

1. Fine-tuning adapts a pre-trained model to specific tasks.
2. PEFT methods like LoRA and QLoRA are popular.
3. Requires high-quality, well-formatted dataset.
4. Evaluate using both automatic metrics and manual testing.
5. Choose the right method based on cost, data size and accuracy.

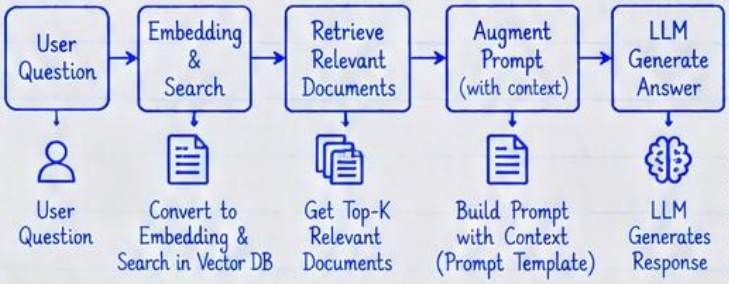
Rahul Yadav

9. Retrieval Augmented Generation (RAG)

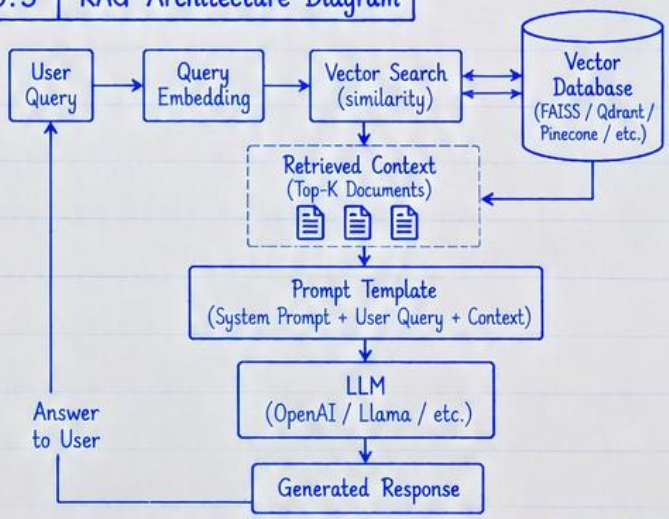
9.1 What is RAG ?

- RAG (Retrieval Augmented Generation) combines the power of LLMs with external knowledge sources.
- Instead of relying only on pre-trained knowledge, it retrieves relevant information from a database or documents and then generates a more accurate response.
- Helps in reducing hallucinations and provides up-to-date information.
- Commonly used for chat with documents, enterprise search and knowledge assistants.

9.2 How RAG Works ? (High Level Flow)



9.3 RAG Architecture Diagram



9.4 Key Components of RAG

Component	Description
Data Source	Documents (PDF, DOC, Web pages, Database, etc.)
Text Processing	Split text into smaller chunks, clean and preprozess.
Embeddings	Convert text chunks to numerical vectors using embedding models (e.g., OpenAI, SentenceTransformers).
Vector Database	Store embeddings and enable similarity search (e.g., FAISS, Qdrant, Pinecone).
Retriever	Find the most relevant chunks based on user query.
Prompt Template	Combine user query with retrieved context and instructions.
LLM	Generate final response using the augmented prompt.

9.5 Chunking Strategies

Strategy	Description	Pros	Cons
Fixed Size	Split text into fixed number of characters or tokens.	Simple, fast.	May break sentences.
Recursive Character	Split by separators (e.g., paragraphs, headings).	Preserves meaning.	Slightly more complex.
Semantic Chunking	Split based on semantic similarity.	Better context and accuracy.	Requires advanced techniques.

9.6 Embedding Models (Popular)

Model	Key Features
OpenAI (text-embedding-3)	High quality, good for general use.
SentenceTransformers (all-MiniLM-L6-v2)	Open source, fast, good performance.
BGE (BAAI)	State-of-the-art for retrieval tasks.
E5 (Microsoft)	Good for enterprise search.
Gohere Embeddings	Good performance and multi-lingual.

9.7 Vector Databases

Database	Key Features
FAISS (Meta)	Open source, high performance, in-memory.
Qdrant	Open source, supports filtering, payloads.
Pinecone	Managed cloud service, easy to use.
Weaviate	Open source, GraphQL API, hybrid search.
Chroma	Lightweight, easy setup, good for small projects.

9.8 Example: RAG Prompt Template

System Prompt:
You are a helpful assistant. Use the following context to answer the question. If the answer is not in the context, say "I don't know".

Context:
{retrieved_documents}

User Question:
{user_query}

9.9 Use Cases of RAG

- Chat with PDF documents
- Enterprise knowledge base search
- Customer support automation
- Research and academic assistance
- Internal company documentation
- Legal, medical and financial document Q&A

9.10 Key Takeaways

1. RAG improves LLM responses using external knowledge.
2. It uses embeddings and vector search to find relevant context.
3. Helps reduce hallucinations and provides up-to-date information.
4. Choice of embedding model, chunking strategy and vector DB is important for good results.
5. Widely used in real-world applications for document-based Q&A.

Rahul Yadav

10. Evaluation of LLM Applications

10.1 Why Evaluate LLMs ?

- Evaluate helps to measure the quality, reliability and safety of LLM outputs.
- LLMs can sometimes generate incorrect, hallucinated or irrelevant answers.
- Evaluation helps in choosing the right model, prompt and techniques.
- Continuous evaluation is important for production applications.

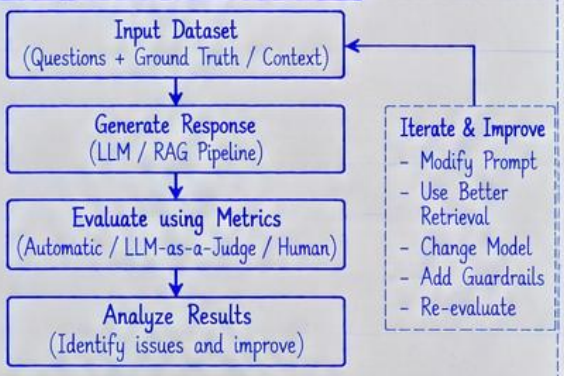
10.2 Key Evaluation Dimensions

Dimension	Description
Accuracy	Is the answer correct and factually valid?
Relevance	Does the answer address the question?
Faithfulness	Is the answer supported by provided context?
Completeness	Does it cover all important aspects?
Fluency	Is the language natural and easy to understand?
Safety	Does it avoid harmful, biased or sensitive content?
Consistency	Does the model give similar answers for same input?

10.3 Types of Evaluation

Type	Description	Use Case
Automatic Evaluation	Uses metrics and models to evaluate answers.	Fast, scalable, cost-effective.
Human Evaluation	Manual review by humans.	High accuracy, complex cases.
LLM as a Judge	Use another LLM to evaluate the response.	Good for scalability.
Hybrid Evaluation	Combination of automatic and human evaluation.	Best for production systems.

10.4 Evaluation Workflow



10.5 Common Evaluation Metrics

Metric	Description	Use Case
Exact Match	Predicted answer exactly matches ground truth.	Simple Q&A, factual tasks.
F1 Score	Overlap of tokens between prediction and ground truth.	Extraction, QA.
BLEU / ROUGE	N-gram based similarity.	Text summarization.
BERTScore	Uses embeddings to measure semantic similarity.	Open-ended generation.
LLM as a Judge	Use another LLM to score the answer (1-5 or pass/fail).	General evaluation.
Faithfulness Score	Checks if answer is supported by context.	RAG applications.
Relevance Score	Measures if answer is on topic.	Chatbots, search.
Toxicity Score	Detects harmful, biased or unsafe content.	Safety evaluation.

10.6 Example: LLM as a Judge

Prompt Template:

```
You are an expert evaluator.
Given a question, an expected answer and a model answer, rate the model answer from 1 to 5 based on accuracy, relevance and completeness.

Question: {user_query}
Expected Answer: {ground_truth}
Model Answer: {model_output}

Return only the score and a short reason.
```

Example Output:

```
Score: 4
Reason: The answer is mostly correct and relevant, but missing some details about edge cases.
```

10.7 Evaluation Dataset

- Use a diverse set of questions covering different scenarios.
- Include both simple and complex questions.
- For RAG, keep ground truth answers or relevant context.
- Use real user queries or benchmark datasets.
- Split dataset into test and validation sets.

10.8 Tools for Evaluation

- LangChain Evaluation (LLM as a Judge, QA, criteria based)
- Ragas (for RAG specific metrics)
- DeepEval (LLM evaluation framework)
- OpenAI / Anthropic Evals
- Custom Python scripts (using BLEU, ROUGE, BERTScore, etc.)

10.9 Best Practices

1. Use multiple metrics (not just one).
2. Evaluate on real-world and edge case data.
3. Use a combination of automatic and human evaluation.
4. Continuously monitor model performance in production.
5. Track and analyze failure cases (hallucination, irrelevant answers, toxicity).
6. Improve using prompt engineering, better retrieval, fine-tuning or model upgrade.

10.10 Key Takeaways

1. Evaluation is essential for building reliable LLM applications.
2. Use metrics like Exact Match, BERTScore and LLM as a Judge.
3. RAG applications require faithfulness and relevance evaluation.
4. Combine automatic and human evaluation for best results.
5. Continuous evaluation and iteration improve the application over time.

Rahul Yadav

11. LangChain - Framework Overview

11.1 What is LangChain ?

- LangChain is a framework for building LLM applications using models, prompts, retrieval, tools and memory.
- It provides building blocks and abstractions to orchestrate different components.
- It is not an LLM itself, but a framework that helps you connect LLMs with external data, tools and workflows.
- Helps in building real world applications like chatbots, document Q&A, agents, etc.

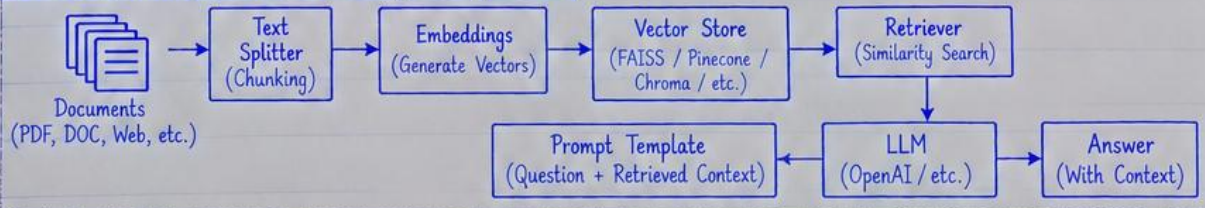
11.2 Core Building Blocks

Chat Models	Interface to different LLM providers (OpenAI, Anthropic, etc.)
Prompt Templates	Create reusable and dynamic prompts.
Output Parsers	Convert model output into structured format (e.g., JSON, Pydantic).
Retrievers / Vector Stores	Retrieve relevant documents using embeddings (e.g., FAISS, Pinecone, Chroma).
Tools	Tools for external actions (search, APIs, databases, code execution, etc.).
Agents	Use LLM to decide which tool to use and perform actions.
Chains / Runnables	Combine multiple steps into a workflow using LCEL (LangChain Expression Language).

11.3 Basic Workflow of LangChain



11.4 RAG Workflow using LangChain



11.5 Simple LangChain Code Example

```
from langchain_core.prompts import ChatPromptTemplate

# Create a prompt template
prompt = ChatPromptTemplate.from_template(
    "Explain {topic} simply."
)
# Assume model is initialized separately
# e.g. model = ChatOpenAI(model="gpt-3.5-turbo")

# Create a chain using LCEL (pipe operator)
chain = prompt | model

# Invoke the chain
result = chain.invoke({"topic": "LLMs"})
print(result)
```

- Model (e.g., ChatOpenAI, ChatAnthropic, etc.) must be initialized separately.
- LangChain APIs may vary depending on the version and provider.

11.6 LCEL / Runnable (Chain Composition)

- LangChain Expression Language (LCEL) allows us to chain components using the pipe operator (|).
- Each component is a Runnable (e.g., prompt, model, parser, retriever, etc.).
- It makes the code simple, modular and readable.
- Supports streaming, batch processing and async execution (depending on the runnable).



11.7 Integrations

Category	Examples
Model Providers	OpenAI, Anthropic, Cohere, HuggingFace
Vector Databases	FAISS, Pinecone, Chroma, Weaviate, Qdrant
Document Loaders	PDF, DOCX, Web Pages, Notion, YouTube, etc.
Tools	SerpAPI (Web search), APIs, Database, Python REPL
Other	LangSmith (tracing), Streamlit, FastAPI, etc.

11.8 Best Practices

1. Keep API keys and secrets in environment variables.
2. Use proper document chunking and embeddings.
3. Validate and sanitize model outputs.
4. Add tracing and evaluation (e.g., LangSmith).
5. Handle errors, retries and token limits.
6. Test for cost, latency and response quality.

11.9 Key Takeaways

1. LangChain helps connect LLMs with data, tools and workflows.
2. It provides modular building blocks (prompts, models, retrievers, tools, agents, etc.).
3. Use retrieval (RAG) to work with your private knowledge.
4. Agents and tools enable LLMs to take real actions.
5. Follow best practices and evaluate for cost, latency and quality.

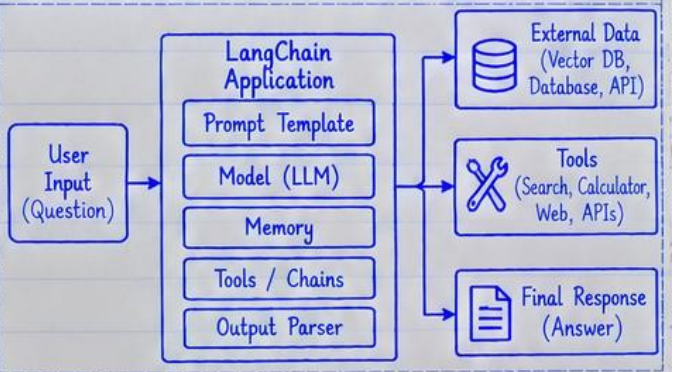
Rahul Yadav

12. Building LLM Applications with LangChain

12.1 What are LLM Applications ?

- LLM applications use Large Language Models with external data, tools and workflows to solve real-world problems.
- LangChain makes it easy to build such applications by providing reusable components.
- Examples: Chatbots, Document Q&A, Agents, Summarization, Code Assistants, Data Analysis, etc.

12.2 Architecture of an LLM Application



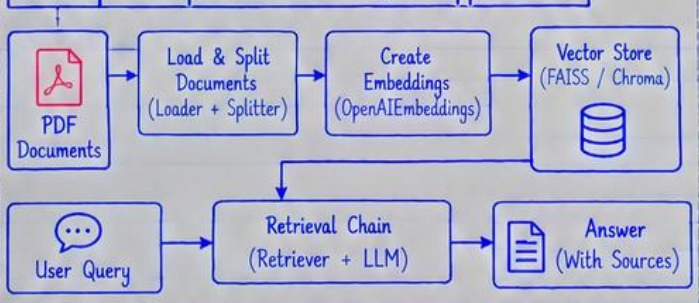
12.3 Common Use Cases

Use Case	Description
Chatbot	Conversational assistant with memory.
Document Q&A	Ask questions on your private documents using RAG.
Data Analysis	Use tools like Python to analyze data.
Code Assistant	Generate, explain and debug code.
Agent-based App	Use tools and APIs to take actions.
Summarization	Summarize long documents or videos.

12.4 Steps to Build an LLM Application

1. Select the use case (e.g., document Q&A).
2. Choose the model (OpenAI, Anthropic, etc.).
3. Prepare and load the data (if needed).
4. Create embeadings and store in vector DB.
5. Build chains or use LangChain Expression Language (LCEL).
6. Add memory (optional).
7. Integrate tools (optional).
8. Parse and format the output.
9. Deploy the application (Streamlit, FastAPI, etc.).

12.5 Example: Document Q&A Application



12.6 Code Example (End-to-End)

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chat_models import ChatOpenAI
from langchain.chains import RetrievalQA

# Load document
loader = PyPDFLoader("data.pdf")
docs = loader.load()

# Split documents
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200)
splits = splitter.split_documents(docs)

# Create embeddings and store in FAISS
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.from_documents(splits, embeddings)

# Create retrieval chain
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
chain = RetrievalQA.from_chain_type(
      llm=llm, retriever=vectorstore.as_retriever()
)
# Ask a question
query = "What is the main topic of this document?"
result = chain.invoke({"query": query})
print(result["result"])
```

12.7 Useful LangChain Compoments

Component	Purpose
Document Loaders	Load data from PDF, websites, databases, etc.
Text Splitters	Split documents into smaller chunks.
Embeddings	Convert text to vector representations.
Vector Stores	Store and search embeddings (FAISS, Chroma, etc.).
Chains	Combine multiple steps (e.g., RetrievalQA).
Memory	Maintain conversation history.
Tools	Use external tools (search, calculator, APIs).
Agents	Decide which tool to use and take actions.
Output Parsers	Format and parse model outputs.

12.8 Deployment Options

Option	Description
Streamlit	Quick web UI for prototype and demo.
FastAPI	Build REST APIs for production.
Docker	Containerize the application.
Cloud (AWS/GCP)	Deploy for scalable and production use.

12.9 Key Takeaways

1. LangChain simplifies building real-world LLM applications.
2. Use modular components like loaders, chains, memory and tools.
3. RAG with LangChain is powerful for document-based Q&A.
4. You can build chatbots, agents, data analysis and more.
5. Deploy your application using Streamlit, FastAPI or cloud.

Rahul Yadav

13. Tools, Agents and Function Calling

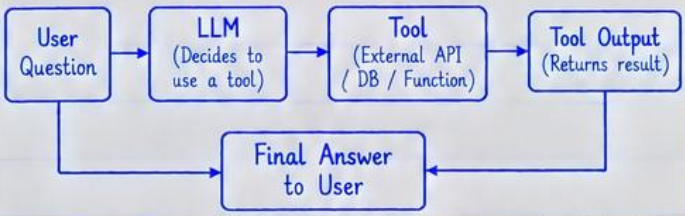
13.1 What are Tools in LangChain ?

- Tools allow LLMs to interact with external systems like search engines, databases, APIs, calculators, etc.
- LLMs can decide when to use a tool based on the user's query.
- Tools help LLMs get real-time information, perform actions and reduce hallucinations.
- LangChain provides a simple way to create and integrate custom tools.

13.2 Types of Tools

Tool Type	Description	Examples
Built-in Tools	Pre-built tools in LangChain.	SerpAPI, Wikipedia, SQLDatabase, etc.
Custom Tools	User-defined tools for specific use cases.	Calculator, Weather API, Internal API, etc.
Vector Store Tools	Search over your documents.	RetrievalQA tool
API Tools	Call external services.	REST APIs (e.g., Booking, Payment)
Utility Tools	General purpose tools.	Python REPL, Shell, File System, etc.

13.3 How Tool Calling Works ?



13.4 Example: Simple Tool (Calculator)

```
from langchain.tools import Tool

def calculator(expression: str) -> str:
    try:
        return str(eval(expression))
    except Exception as e:
        return f"Error: {str(e)}"

calculator_tool = Tool(
    name="calculator",
    func=calculator,
    description="Useful for performing mathematical calculations."
)
```

13.5 Using Tools with an Agent

```
from langchain.agents import initialize_agent, AgentType
from langchain.chat_models import ChatOpenAI

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

tools = [calculator_tool]

agent = initialize_agent(
    tools,
    llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)

response = agent.run("What is 25 * 4?")
print(response)
```

13.6 Agent Types in LangChain

Agent Type	Description
ZERO_SHOT_REACT_DESCRIPTION	Uses tool descriptions to decide actions (most common).
OPENAI_FUNCTIONS	Uses OpenAI function calling (more reliable).
STRUCTURED_CHAT_ZERO_SHOT_REACT	Good for structured input/output.
CONVERSATIONAL_REACT_DESCRIPTION	Maintains conversation history.
SELF_ASK_WITH_SEARCH	Breaks complex questions into simpler ones and uses search.

13.7 Example: Web Search Tool (SerpAPI)

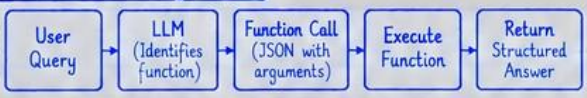
```
from langchain.tools import SerpAPIWrapper
from langchain.agents import load_tools

search = SerpAPIWrapper(serpapi_api_key="YOUR_API_KEY")
tools = load_tools(["serpapi"], serpapi_api_key="YOUR_API_KEY")
agent = initialize_agent(tools, llm, agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION)
response = agent.run("Who is the CEO of OpenAI?")
```

13.8 Function Calling (OpenAI)

- Function calling allows the model to call structured functions with specific parameters.
- The LLM returns a JSON with function name and arguments.
- Useful for reliable and structured outputs.
- LangChain supports function calling through tools and OpenAI function agents.

Function Calling Flow



13.9 Example: Weather Tool with Function Calling

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather for a given city."""
    # Call real weather API (e.g., OpenWeatherMap)
    return f"The current weather in {city} is 28°C and sunny."
```

13.10 Real-world Tool Examples

Use Case	Example Tools
Web Search	SerpAPI, Tavily
Database Queries	SQLDatabaseToolkit
Document Search	RetrievalQA Tool
Code Execution	PythonREPLTool
File Operations	FileSystemTool
Calendar / Email	Custom API Tools (Google, Outlook)
Business APIs	Custom REST API Tools
Data Analysis	Pandas, CSV Tool, DataFrame Tool

13.11 Key Takeaways

1. Tools extend LLM capabilities beyond text generation.
2. Agents can decide which tool to use based on user query.
3. Function calling provides structured and reliable outputs.
4. LangChain makes it easy to integrate and use tools.
5. Useful for real-world applications like chatbots, automation, data analysis and more.

Rahul Yadav

14. Evaluation of LLMs

14.1 What is LLM Evaluation ?

- Evaluation helps to measure the quality, reliability and safety of LLM outputs.
- It ensures the model gives correct, relevant, helpful and safe answers.
- Evaluation is important for selecting the right model, comparing models and improving performance.
- It can be done using automatic metrics, human feedback and LLM-as-a-judge techniques.

14.2 Why Evaluation is Important ?

1. Check if the answer is correct and factually valid.
2. Ensure the response is relevant to the question.
3. Detect hallucinations and incorrect information.
4. Compare different models and prompts.
5. Measure performance on domain-specific tasks.
6. Improve prompts, retrieval and fine-tuning.
7. Ensure safe, unbiased and responsible outputs.
8. Important for production and enterprise use cases.

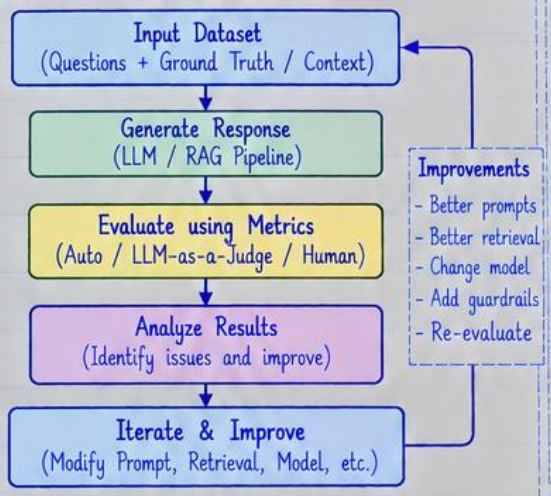
14.3 Evaluation Dimensions

Dimension	Description
Accuracy	Is the answer factually correct?
Relevance	Does it address the user question?
Faithfulness	Is the answer based on provided context?
Completeness	Does it cover all important points?
Fluency	Is the language natural and easy to read?
Safety	Is there any harmful, biased or sensitive content?
Consistency	Does the model give similar answers for similar inputs?

14.4 Types of Evaluation

Type	Description
Automatic Evaluation	Uses metrics and models to evaluate answers.
Human Evaluation	Manual review by humans.
LLM as a Judge	Use another LLM to evaluate the response.
Task-specific Evaluation	Evaluate based on specific use case (e.g., RAG, QA, summarization).
Red Teaming / Safety	Test for harmful, biased or unsafe content.

14.5 Evaluation Workflow



14.6 Common Evaluation Metrics

Metric	Description	Use Case
Exact Match	Predicted answer exactly matches ground truth.	Simple QA, factual tasks.
F1 Score	Overlap of tokens between prediction and ground truth.	QA, information extraction.
BLEU / ROUGE	N-gram based similarity.	Text summarization.
BERTScore	Uses embeddings to measure semantic similarity.	Open-ended generation.
LLM as a Judge	Use another LLM to score the answer (1-5 or pass/fail).	General evaluation.
Faithfulness Score	Checks if answer is supported by context.	RAG applications.
Relevance Score	Measures if answer is on topic.	Chatbots, search.
Toxicity Score	Detects harmful or unsafe content.	Safety evaluation.

14.7 Dataset Preparation

- Use a diverse set of questions covering different scenarios.
- Include both simple and complex questions.
- For RAG, keep ground truth answers or relevant context.
- Use real user queries or benchmark datasets.
- Split dataset into test and validation sets.

14.8 Example: LLM as a Judge

Prompt Template:

```
You are an expert evaluator.
Given a question, an expected answer and a model answer, rate the model answer from 1 to 5 based on accuracy, relevance, completeness and helpfulness.
Question: {user_query}
Expected Answer: {ground_truth}
Model Answer: {model_output}
Return only the score and a short reason.
```

14.9 Example Output

```
Score: 4
Reason: The answer is mostly correct and relevant, but missing some details about edge cases.
```

14.10 Best Practices

1. Use multiple metrics (not just one).
2. Evaluate on real-world and edge case data.
3. Use a combination of automatic and human evaluation.
4. Continuously monitor model performance in production.
5. Track and analyze failure cases (hallucination, irrelevant answers).
6. Improve using prompt engineering, better retrieval and fine-tuning.

Rahul Yadav

15. RAG vs Fine-tuning vs Prompting (Comparison)

15.1 Introduction

- There are multiple ways to make LLMs more useful for real-world applications.
- Prompting uses the existing model as-is with good instructions.
- RAG (Retrieval Augmented Generation) adds external knowledge from your data.
- Fine-tuning additionally trains the model on your specific data.
- Each approach has different use cases, advantages and limitations.

15.2 High Level Comparison

Aspect	Prompting	RAG	Fine-tuning
Uses External Data	No	Yes	Yes (during training)
Model Training	No	No	Yes
Best For	General tasks	Up-to-date private data	Domain-specific specialized tasks
Cost	Low	Medium	High
Latency	Low	Medium	Low (after training)
Data Freshness	Limited	High	Medium
Implementation	Simple	Moderate	Complex

15.3 Prompting

- Prompting means giving clear instructions to the LLM using natural language.
- No external data or model training is used.
- Useful for general knowledge, reasoning, summarization and simple tasks.
- Works well with well-crafted prompts (zero-shot, few-shot, chain-of-thought).

Example:

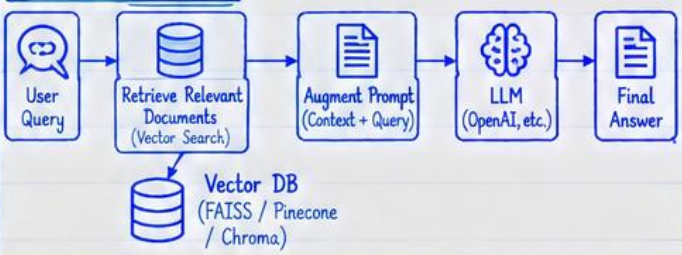
Prompt: "Explain how Spring Boot works in simple words."



15.4 Retrieval Augmented Generation (RAG)

- RAG retrieves relevant information from external sources (e.g., your documents, database, APIs) and provides it to the LLM in the prompt.
- Helps generate accurate and up-to-date answers.
- Useful for private data, documents, enterprise knowledge and frequently changing information.
- Does not require model training.

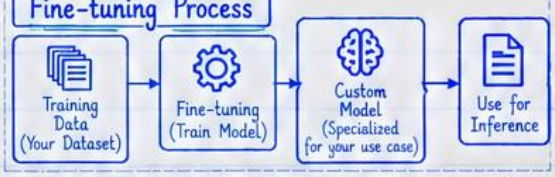
RAG Architecture



15.5 Fine-tuning

- Fine-tuning means training the model on your specific data.
- The model learns domain-specific patterns, format, tone and style.
- Useful when you have large labeled data and need consistent, specialized behavior.
- More complex and expensive compared to prompting and RAG.

Fine-tuning Process



15.6 Detailed Comparison

Criteria	Prompting	RAG	Fine-tuning
Data Source	None	External (Your Data)	Training Dataset
Training Required	No	No	Yes
Accuracy (Private Data)	Low	High	High
Handles Latest Data	No	Yes	Only if re-trained
Domain Customization	Limited	Good	Excellent
Cost	Low	Medium	High
Development Time	Small	Moderate	Large
Flexibility	High	High	Medium
Typical Use Cases	General Q&A, Summarization	Enterprise Knowledge, Docs	Domain-specific Applications

15.7 When to Use What ?

Scenario	Recommended Approach
General questions (no private data)	Prompting
Use your own documents / data	RAG
Need consistent domain-specific style	Fine-tuning
Frequently changing data	RAG
Limited budget and quick prototype	Prompting
High accuracy with private knowledge	RAG or Fine-tuning
Specialized chatbot with fixed behavior	Fine-tuning

15.8 Key Takeaways

1. Prompting is simple and good for general tasks.
2. RAG brings your own data and improves accuracy.
3. Fine-tuning is best for specialized, domain-specific needs.
4. RAG is the most practical approach for enterprise use cases.
5. Choose the right approach based on your data, cost, accuracy and maintenance needs.

Rahul Yadav

16. Real-world Use Cases of LLMs

16.1 Introduction

- LLMs are transforming multiple industries by automating tasks, improving productivity and enabling new applications.
- They can understand, generate and reason on human language, which makes them useful in many real-world scenarios.
- From chatbots to document analysis, LLMs are now used in almost every domain.
- In this section, we will explore practical use cases with examples.

16.2 Major Application Areas

1. Chatbots and Customer Support
2. Document Processing and Summarization
3. Search and Knowledge Management
4. Code Generation and Developer Productivity
5. Data Analysis and Insights
6. Healthcare and Life Sciences
7. Finance and Risk Analysis
8. Education and Learning
9. Content Creation and Marketing
10. Industry-specific Use Cases

16.3 Chatbots and Customer Support

- LLMs can handle customer queries, provide instant support and resolve common issues.
- They can understand natural language, maintain conversation history and give human-like responses.
- They reduce support costs and improve customer satisfaction.



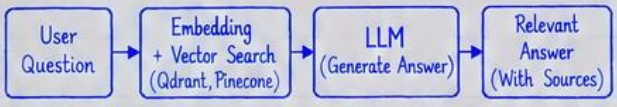
16.4 Document Processing and Summarization

- LLMs can read, understand and summarize long documents like PDFs, reports, contracts, etc.
- Useful for legal, research, HR and enterprise knowledge.
- They can also extract key information, answer questions and convert unstructured data into structured format.



16.5 Search and Knowledge Management

- LLMs combined with vector databases (RAG) can search and retrieve information from your private data (documents, wiki, databases, etc.).
- Helps employees quickly find the right information.
- Useful for internal tools, knowledge bases and enterprise search.



16.6 Code Generation and Developer Productivity

- LLMs can write, explain, debug and optimize code in multiple programming languages.
- They act as AI pair programmers.
- Useful for faster development, learning new technologies and code reviews.



16.7 Healthcare and Life Sciences

- LLMs help in medical research, patient data analysis and generating medical summaries.
- They can assist doctors by providing relevant information.
- Useful for drug discovery, clinical trials, medical literature search and patient support.



16.8 Finance and Risk Analysis

- LLMs can analyze financial reports, detect risks, summarize earnings calls and answer domain-specific questions.
- Useful for fraud detection, compliance, investment research and report generation.



16.9 Education and Learning

- LLMs can act as personal tutors, explain complex topics, generate practice questions and provide step-by-step solutions.
- Useful for students, educators and professional learning platforms.



16.10 Content Creation and Marketing

- LLMs can generate blog posts, social media content, email templates, product descriptions and more.
- They help in ideation, editing and improving content quality.



Rahul Yadav

17. Limitations, Risks and Best Practices

17.1 Limitations of LLMs

- Can generate incorrect or hallucinated information.
- Limited real-time knowledge (depends on training data).
- Struggles with highly specialized or niche domain knowledge.
- May produce biased, toxic or inappropriate content.
- Not always good at complex reasoning or mathematical calculations.
- Requires careful prompt design and evaluation.
- Large models can be expensive to run.

17.2 Common Risks

Risk	Description
Hallucination	Generates incorrect or fake information that sounds convincing.
Bias	May produce biased or unfair responses from training data.
Toxic Output	Can generate harmful, offensive or inappropriate content.
Data Privacy	Risk of exposing sensitive or confidential data in prompts.
Security Risks	Prompt injection and data leakage attacks.
Misinformation	May provide outdated or incorrect facts.
Over-reliance	Users may blindly trust AI outputs.
Compliance	Legal and regulatory issues in industries like finance, healthcare, etc.

17.3 Challenges in Real-world Use

- Ensuring factual accuracy and reliability.
- Handling domain-specific knowledge.
- Integrating with real-time and live data.
- Managing cost and latency for large-scale usage.
- Evaluating and monitoring model performance.
- Dealing with ambiguous or unclear user queries.
- Ensuring data privacy and security.
- Building guardrails to prevent misuse.

17.4 Best Practices (General)

1. Use clear and specific prompts.
2. Provide relevant context and examples.
3. Use retrieval (RAG) for up-to-date and domain data.
4. Validate and verify important outputs.
5. Apply guardrails and content filters.
6. Monitor and log model usage.
7. Continuously evaluate and fine-tune.
8. Follow data privacy and compliance guidelines.
9. Use the right model for the right use case.
10. Keep humans in the loop for critical decisions.

17.5 Safety and Guardrails

- Use content moderation filters (e.g., OpenAI Moderation API).
- Implement input and output validation.
- Restrict sensitive topics or restricted data.
- Use role-based access and authentication.
- Set clear usage policies for your application.
- Monitor logs and detect abuse or unusual activity.

17.6 Prompt Engineering Tips

- Be clear, concise and specific.
- Provide examples (few-shot) when needed.
- Break complex tasks into smaller steps.
- Use structured formats (e.g., JSON) for reliable output.
- Use system prompts to set the behavior.
- Specify output style, tone and constraints.
- Iterate and refine prompts based on results.

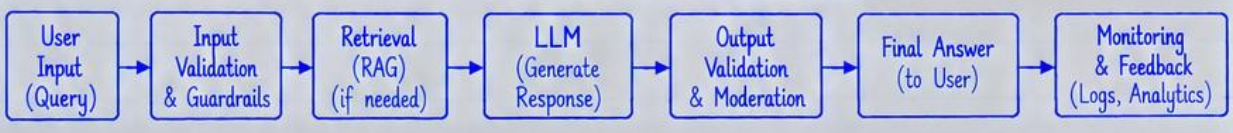
17.7 Evaluation and Monitoring Best Practices

- Use a combination of automatic metrics and human review.
- Test with diverse and real-world datasets.
- Monitor for hallucinations, bias and unsafe content.
- Track user feedback and model performance over time.
- Set up alerts for unusual or harmful outputs.
- Regularly update prompts, retrieval data and models.
- Perform A/B testing for different prompts or models.

17.8 Summary

- LLMs are powerful but not perfect.
- Understand their limitations and risks.
- Follow best practices to use them safely and effectively.
- Use guardrails, evaluation and monitoring.
- Choose the right approach (Prompting, RAG or Fine-tuning) for your use case.
- Always verify critical information and keep humans in the loop.

17.9 Recommended Workflow for Safe and Reliable LLM Applications



Rahul Yadav

18. Mini Project (End-to-End LLM App)

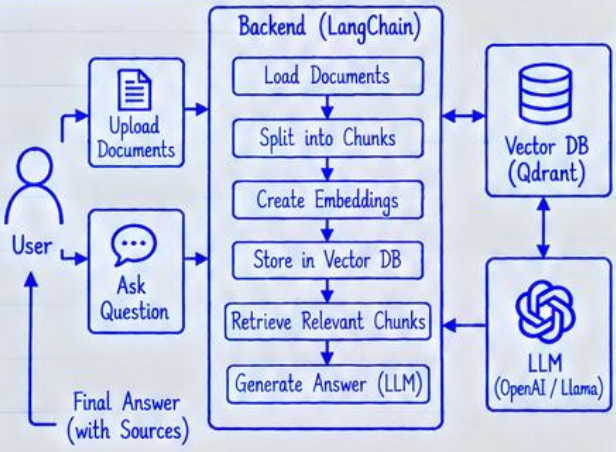
18.1 Project Overview

- Build a complete end-to-end LLM application using real-world technologies.
- Users can upload documents (PDF, DOC, TXT), chat with their data and get accurate answers.
- We will use RAG (Retrieval Augmented Generation) with a vector database.
- Build a simple, clean UI (Streamlit) and deploy locally (or on cloud).
- This project combines everything we learned: prompting, embeddings, vector DB, RAG, tools and LLM evaluation basics.

18.2 Tech Stack

Layer	Technologies
Frontend (UI)	Streamlit (Python)
Backend / Orchestration	LangChain
LLM	OpenAI (GPT-3.5/GPT-4) or Llama (Ollama)
Embedding Model	SentenceTransformers
Vector Database	Qdrant (or FAISS / Pinecone)
Document Loader	LangChain Document Loaders
Text Splitter	RecursiveCharacterTextSplitter
RAG Chain	ConversationalRetrievalChain
Deployment (Optional)	Streamlit Cloud / Render / AWS
Others	PyPDF2, python-docx, tiktoken, dotenv etc.

18.3 High Level Architecture



18.4 Project Features

1. Upload multiple documents (PDF, DOC, TXT)
2. Automatically split and index documents
3. Store embeddings in Qdrant
4. Chat with your documents using natural language
5. Show source documents for the answer
6. Maintain chat history (Conversational RAG)
7. Simple and clean Streamlit UI
8. Optional: Deploy on cloud

18.5 Folder Structure

```
llm-mini-project/
├── app.py              # Streamlit UI
├── rag_pipeline.py     # Load, split, embed, retrieve, generate
├── utils.py            # Helper functions (PDF loader, etc.)
├── requirements.txt    # Python dependencies
├── .env                # API keys (OpenAI, etc.)
└── data/               # Uploaded documents (optional)
```

18.6 Key Components with Code Snippets

1. Load Documents

```
from langchain.document_loaders import PyPDFLoader
loader = PyPDFLoader("sample.pdf")
docs = loader.load()
```

2. Split Documents

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
chunks = splitter.split_documents(docs)
```

3. Create Embeddings and Store in Qdrant

```
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Qdrant
embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
vectordb = Qdrant.from_documents(
            documents=chunks,
            embedding=embedding,
            location=":memory:"  # or your Qdrant URL
)
```

4. Create RAG Chain

```
from langchain.chains import ConversationalRetrievalChain
from langchain.chat_models import ChatOpenAI
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
retriever = vectordb.as_retriever(search_kwargs={"k": 4})
chain = ConversationalRetrievalChain.from_llm(llm, retriever=retriever)
```

18.7 Streamlit UI (app.py)

```
import streamlit as st
from rag_pipeline import chain

st.set_page_config(page_title="LLM RAG App")
st.title("📄 Chat with Your Documents")

query = st.text_input("Ask a question:")

if query:
    response = chain.run({"question": query})
    st.write("### Answer"
    st.write(respoose)
```

18.8 Example Use Cases

- Company internal document search (HR policies, manuals)
- Research paper summarization
- Legal document analysis
- Customer support knowledge base
- Product documentation chatbot
- Personal learning assistant (your own notes)

18.9 Key Takeaways

1. Combines multiple concepts: embeddings, vector DB, RAG, UI
2. Real-world and practical application
3. Can be extended with features like auth, multi-user, tools
4. Great project for portfolio and interview discussion
5. Helps you understand how LLMs are used in real products

Rahul Yadav

19. Interview Questions and Key Takeaways

19.1 Common Interview Question Topics

1. LLM Basics and Concepts
2. Tokenization and Embeddings
3. Model Architecture (Transformer)
4. Prompting Techniques
5. RAG (Retrieval Augmented Generation)
6. Fine-tuning and Evaluation
7. LangChain and Tools
8. Real-world Use Cases
9. System Design for LLM Applications
10. Best Practices, Limitations and Ethics

19.2 Sample Interview Questions (with Short Answers)

Q1: What are LLMs ?
A: Large Language Models are deep learning models trained on massive text data to understand, generate and reason using human language.

Q2: What is tokenization ?
A: Tokenization converts text into smaller units called tokens which are processed by the model. (e.g., words, subwords)

Q3: What is RAG ?
A: RAG (Retrieval Augmented Generation) retrieves relevant information from external sources (e.g., documents, DB) and uses it in the prompt to generate accurate answers.

Q4: When to use fine-tuning vs RAG ?
A: Use RAG for up-to-date information and domain knowledge. Use fine-tuning when you need specific style, behavior or specialized task performance.

19.3 LangChain Related Questions

Q1: What is LangChain ?
A: LangChain is a framework to build LLM applications using chains, tools, agents, memory and integrations with external systems like vector databases.

Q2: What is a chain in LangChain ?
A: A chain is a sequence of steps (e.g., prompt → LLM → parser) to process inputs and generate outputs.

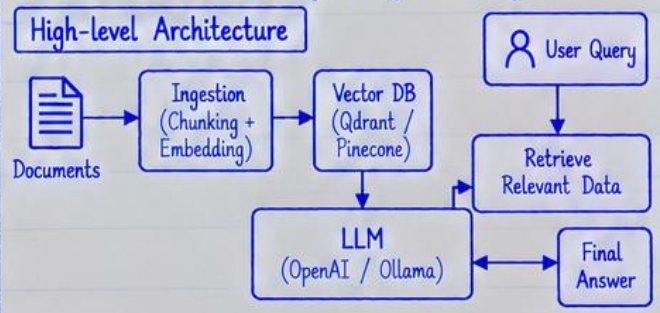
Q3: What is the difference between Chains and Agents ?
A: Chains follow a fixed sequence of steps, while Agents can dynamically decide which tool to use based on the input.

Q4: What are common integrations in LangChain ?
A: Vector DBs (Qdrant, Pinecone), tools (APIs, calculators), loaders (PDF, web) and memory modules.

19.4 System Design Questions

Q1: How would you design a scalable LLM application ?
Q2: How to handle large documents efficiently ?
Q3: How to ensure low latency and high availability ?

High-level Architecture



19.5 Scenario-based Questions

Q1: How would you improve answer accuracy in RAG ?
A: Use better chunking, relevant embeddings, metadata filters, hybrid search and re-ranking.

Q2: How to handle hallucinations ?
A: Use grounding (RAG), clear instructions, system prompts, and verify answers with external sources.

Q3: How to evaluate LLM outputs ?
A: Use metrics like accuracy, relevance, faithfulness, BLEU/ROUGE, BERTScore or human evaluation.

Q4: How to design a cost-effective solution ?
A: Use open-source models (e.g., Llama), caching, smaller models, optimal chunking and efficient prompting.

19.6 Key Takeaways from This Guide

1. Understand the core concepts, architecture and workflow of LLMs.
2. Learn tokenization, embeddings and model architectures.
3. Use prompting, RAG and fine-tuning for real-world applications.
4. Work with tools like LangChain, vector DBs and OpenAI/Ollama.
5. Evaluate model performance and follow best practices.
6. Be aware of limitations, risks and ethical considerations.
7. Build real-world projects to gain hands-on experience.
8. Prepare for interviews with conceptual, practical and system design questions.

19.7 Quick Revision Checklist

- ☐ Basics of LLMs, Tokens and Embeddings
- ☐ Transformer Architecture
- ☐ Prompting Techniques (Zero-shot, Few-shot, CoT)
- ☐ RAG Pipeline and Vector Databases
- ☐ Fine-tuning Concepts
- ☐ LangChain (Chains, Agents, Tools, Memory)
- ☐ Evaluation Metrics
- ☐ Real-world Use Cases
- ☐ Limitations and Risks
- ☐ System Design and Best Practices
- ☐ Interview Questions Practice

19.8 Final Advice for Interviews

- Be clear with fundamentals (don't just memorize).
- Explain concepts with real-world examples.
- Show hands-on project experience (even small projects).
- Solve end-to-end scenarios, not just theory.
- Discuss trade-offs, limitations and best practices.
- Stay updated with latest LLM trends and tools.
- Practice explaining your projects and code.
- Be confident and think out loud.

Rahul Yadav